BOOKSHOP ANALYTICS

Pipeline Big Data de bout en bout

PostgreSQL · Airflow · Snowflake · dbt · Streamlit

BIG DATA

Mamadou Saidou Bah
BOLLO KONDABEKA Jürgen
Fabien MFUAMBA KABASELE
Master 2 Big Data · Avril 2026

Le problème

P

Source transactionnelle

La base PostgreSQL de la librairie est optimisée pour les opérations métier, pas pour l'analyse.

A

Besoin analytique

Agrégats, historisation, croisements inter-tables : impossibles à tenir sur la base de production.

V

Restitution métier

Les équipes veulent des KPI accessibles sans requêter du SQL, mis à jour automatiquement.

Ce que le projet livre

01 Pipeline automatisé

Ingestion, transformations et publication déclenchables en 1 clic ou programmables dans Airflow.

02 Stack cloud moderne

Snowflake pour le stockage analytique, dbt pour des transformations SQL testées et documentées.

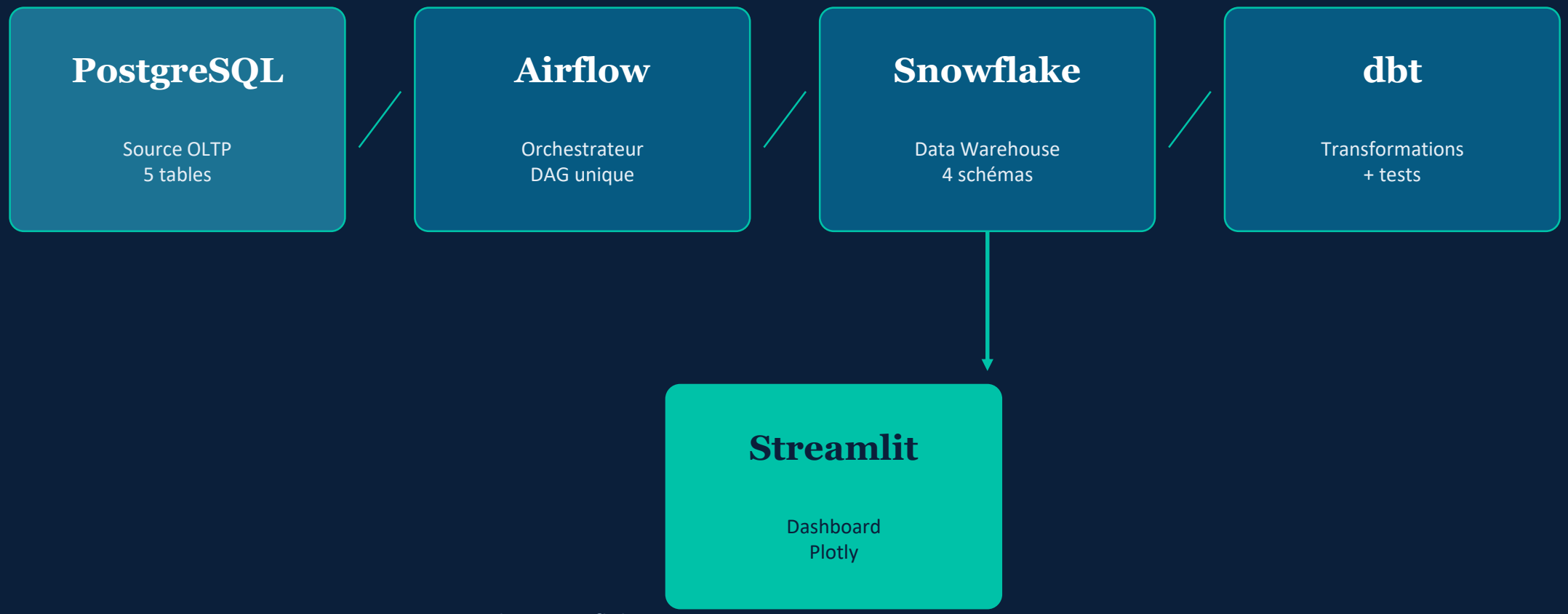
03 Reproductibilité

Tout l'environnement dans Docker Compose : `make up` et l'infrastructure démarre.

04 Observabilité

Logs structurés, tests dbt automatiques, CI GitHub Actions, dépannage documenté.

Flux de bout en bout



Couches Snowflake : RAW → STAGING → WAREHOUSE → MARTS

Technologies retenues

Source transactionnelle	**PostgreSQL 15**	*Standard open-source, conteneurisé*
Orchestration	**Apache Airflow 2.10**	*DAG Python, LocalExecutor*
Data warehouse cloud	**Snowflake**	*Trial 30 jours, AWS eu-west-3*
Transformations SQL	**dbt-core 1.9 + dbt-snowflake**	*Modèles versionnés, tests auto*
Dashboard	**Streamlit + Plotly**	*UI Python interactive*
Infrastructure	**Docker Compose**	*5 services isolés, 1 commande*
CI / Tests	**GitHub Actions + pytest + ruff**	*Lint, tests, dbt parse*

Modèle PostgreSQL

5 tables transactionnelles

- category — catégories de livres
- books — catalogue (ISBN, prix catalogue)
- customers — clients (nom, email, ville, pays)
- factures — entêtes de factures (total, payé)
- ventes — lignes (book_id, qte, pu)

~30 enregistrements de démonstration chargés automatiquement au démarrage

— snippet

```
CREATE TABLE books (
  id             SERIAL PRIMARY KEY,
  category_id    INT REFERENCES category(id),
  code           VARCHAR(32) UNIQUE,
  intitule       VARCHAR(255),
  isbn_10        VARCHAR(10),
  isbn_13        VARCHAR(13),
  prix_catalogue NUMERIC(10,2)
);

CREATE TABLE ventes (
  id          SERIAL PRIMARY KEY,
  facture_id  INT REFERENCES factures(id),
  book_id     INT REFERENCES books(id),
  pu          NUMERIC(10,2),
  qte         INT,
  date_edit   TIMESTAMP
);
```

DAG Airflow · bookshop_pipeline

1

create_bookshop_structures

Crée la base BOOKSHOP + schémas + tables RAW via SQL Snowflake.

2

ingest_postgres_to_snowflake

Lit les 5 tables Postgres, truncate RAW, insert ligne par ligne.

3

ensure_dbt_profile

Prépare profiles.yml à partir du template si absent.

4

dbt_build

Exécute dbt build = run + test sur toutes les couches.

Durée totale d'un run complet : ~35 secondes

Modèle dbt en 3 couches

STAGING

Nettoyage

- stg_books
- stg_category
- stg_customers
- stg_factures
- stg_ventes

Typage, renommage, colonnes normalisées.

WAREHOUSE

Modèle étoile

- dim_books
- dim_category
- dim_customers
- fact_ventes
- fact_factures
- fact_books_mois
- fact_books_annees

Dimensions + faits, agrégats temporels.

MARTS

One Big Table

- obt_sales

Table dénormalisée prête pour le dashboard.

Dashboard Streamlit

8	5	3	100 %
Ventes	**Livres**	**Clients**	**Payé**
lignes de factures	*au catalogue*	*actifs*	*taux de recouvrement*

Visualisations Plotly

- Histogramme du chiffre d'affaires par mois
- Camembert des ventes par livre
- Barres des montants payés par client
- Aperçu tabulaire de la OBT

RÉSILIENCE

✓ Retry exponentiel (3 tentatives)

✓ Timeouts configurables

✓ Cache requête 5 min

✓ Validation env vars au boot

Tests & Intégration Continue

Tests dbt (automatiques)

- not_null sur toutes les clés
- unique sur identifiants métier
- relationships entre dim/fact
- Exécutés à chaque dbt build
- Échec → DAG marqué failed

CI GitHub Actions

- ruff check (lint Python)
- pytest (tests DAG + dashboard)
- dbt parse (validation SQL)
- Déclenché sur push + PR
- Logs publics dans l'onglet Actions

Démonstration live

Scénario (≈ 3 minutes)

01	`docker compose ps`	→ *Les 4 services actifs*
02	`Airflow UI · localhost:8080`	→ *DAG bookshop_pipeline*
03	`Trigger DAG`	→ *4 tâches vertes successives*
04	`Snowsight · BOOKSHOP.MARTS.OBT_SALES`	→ *Données dénormalisées prêtes*
05	`Streamlit · localhost:8501`	→ *KPI + graphiques Plotly*

Ce qui a été livré

~800
lignes de code

4
schémas Snowflake

14
modèles dbt

35 s
par run complet

Points forts

R Reproductible

make up reconstruit tout en une commande

T Testé

tests dbt + pytest + CI GitHub Actions

D Documenté

guides Snowflake / local / troubleshooting

Évolutions envisagées

CDC

Ingestion incrémentale

Remplacer truncate/insert par une capture des changements (Debezium, Snowpipe Streaming).

MON

Monitoring & alerting

Hooks Airflow on_failure vers Sentry / Slack ; dashboards Grafana sur la fraîcheur des données.

SEC

Sécurité production

Auth Snowflake par clé RSA, secrets manager (Vault / AWS SSM), rotation automatique.

DOC

Catalogue de données

dbt docs publiées + intégration Datahub ou OpenMetadata pour la gouvernance.

CLD

Airflow managé

Migration vers MWAA ou Astronomer pour la haute disponibilité.

MERCI

Questions & discussion

Mamadou Saidou Bah
BOLLO KONDABEKA Jürgen
Fabien MFUAMBA KABASELE

Master 2 Big Data · Avril 2026

Code source : github.com/<votre-user>/bookshop-bigdata